

S5. 온체인 이벤트 수신 설계 즉시 처리의 위험과 비동기 분리

202 패턴 · Webhook Receiver · Queue 비동기 적재 · DMZ 파이프라인

2026.05 | M2 S5

M1 완료 → M2 시작

M1 완료

아키텍처 이해 ✓
개발환경 구성 ✓
Fork + TypeChain ✓



M2 시작

DMZ 이벤트 파이프라인
S5~S12 · 8세션

M2의 핵심 질문: VASP 이벤트가 들어왔을 때, 시스템은 어떻게 안전하게 처리하는가?

M2 — DMZ 이벤트 파이프라인 로드맵

S5

이벤트 수신 설계
202 패턴

S6

Redis Streams
내부 구조

S7

Streams
CLI 실습

S8

Webhook 보안
병렬 처리

S9

처리 순서
불변 규칙

S10

EventConsumer
먹등 처리

S11

DLQ 설계
장애 격리

S12

Finalized
복원력 검증

▶ 오늘(S5) = 파이프라인의 시작점 — 이벤트가 어떻게 들어오고, 왜 비동기여야 하는가

S5 — 오늘 배울 핵심 3가지

1

즉시 처리의 위험

DB 장애 시 이벤트 영구 유실
복구 불가 시나리오 이해

2

202 즉시 응답 패턴

Webhook 수신 즉시 202 반환
Queue 적재는 비동기로 분리

3

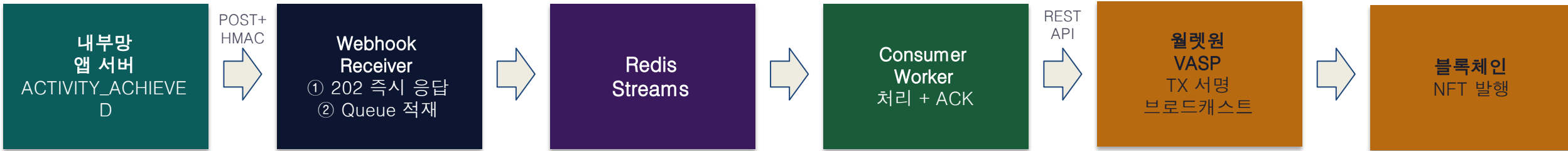
Queue 중간 단계

내구성 보장 · Consumer 독립 확장
재처리 가능한 구조 설계

완료 기준: 202 패턴 설계 이해 + WebhookReceiver 라우트 구조 완성

DMZ — 양방향 파이프라인 허브

방향 1 발행 요청 내부망 → 블록체인 (Phase 1: Consumer → VASP(월렛원) → 블록체인)



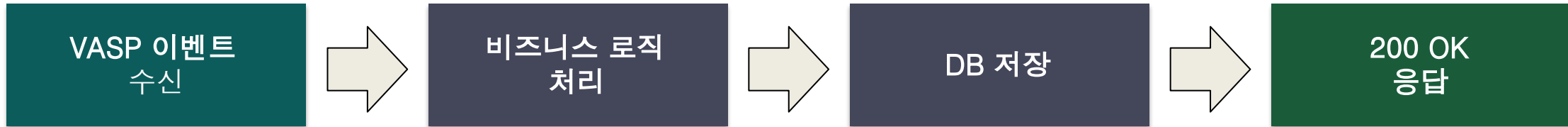
방향 2 결과 수신 블록체인 → 내부망 | Phase 1: 월렛원 Webhook → WebhookReceiver / Phase 2+: ChainEventListener



S5 집중 구간: 방향 1 입구 — WebhookReceiver 202 응답 + Queue 적재 | Phase 1 write: Consumer → 월렛원(VASP) → 블록체인 / 방향 2는 M5 이후

즉시 처리 — 직관적이지만 위험한 설계

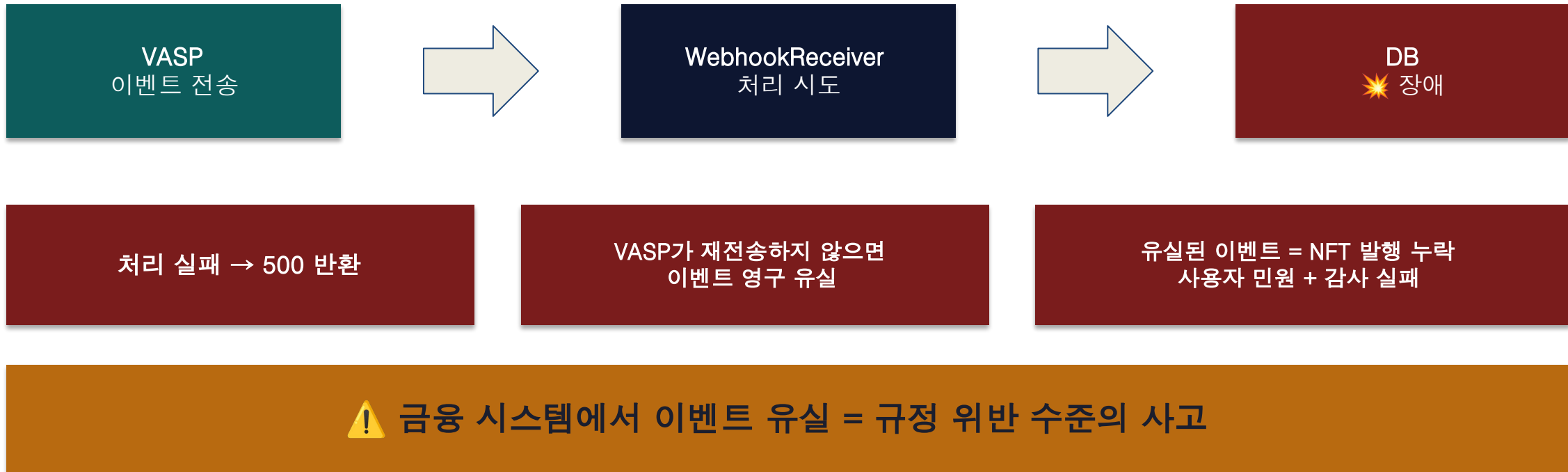
VASP에서 이벤트가 왔다. 바로 DB에 저장하면 안 될까?



↑ 처음 보면 깔끔해 보인다

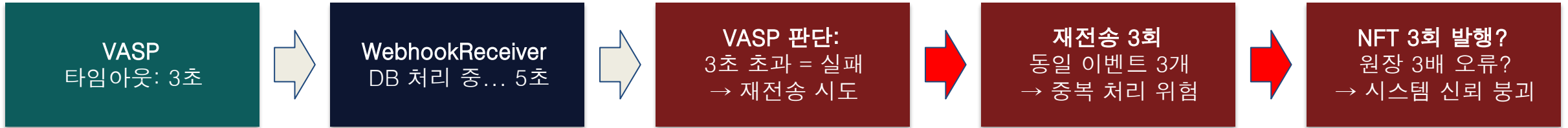
- 문제 1: DB 장애 시 — 이벤트 영구 유실
- 문제 2: VASP 타임아웃 — 처리가 길어지면 VASP가 재전송 또는 포기
- 문제 3: 단일 장애점 — DB, 로직, 응답이 하나의 트랜잭션에 묶임

시나리오 — DB 장애 시 이벤트 영구 유실



VASP 계약 조건에 따라 재전송 보장이 없는 경우도 있음 — 우리 시스템이 내구성을 가져야 한다

시나리오 — VASP 타임아웃으로 인한 재전송 폭탄



해결책: 즉시 202 반환 → VASP는 성공으로 인식 → 재전송 없음

처리는 비동기로 — Queue에 적재만 하고 응답은 즉시

즉시 처리의 3가지 치명적 약점

 이벤트 유실

DB 장애 → 이벤트 사라짐
VASP 재전송 보장 없으면 복구 불가

 중복 처리

VASP 타임아웃 → 재전송
멥등성 없으면 중복 발행

 단일 장애점

처리·DB·응답 하나로 묶임
부분 장애가 전체 장애로 전파

해결 방향: 수신 ↔ 처리 분리

→ 수신: 202 즉시 응답 / 처리: Queue 기반 비동기

설계 원칙 — 수신과 처리를 분리하라

수신 레이어

- POST /webhook 엔드포인트
- 이벤트 유효성 검사 (기본)
- Queue에 적재
- 즉시 202 반환
- DB 접근 ❌ 금지



처리 레이어

- Queue에서 이벤트 읽기
- 비즈니스 로직 실행
- DB 저장 (원장 업데이트)
- 멍등성 보장
- XACK (완료 선언)

수신은 빠르고 단순하게 / 처리는 안전하고 재시도 가능하게

Queue = 내구성 버퍼

내구성

이벤트를 메모리에 보관하면
서버 재시작 시 유실
Queue는 영구 저장

독립 확장

Consumer 늘려도
Webhook 수신에 영향 없음
수신·처리 독립 스케일링

재처리

처리 실패 시 Queue에서
다시 읽기 가능
DLQ로 격리 후 재큐잉

교보 프로젝트: Redis Streams를 Queue로 사용 — S6에서 상세 학습

Queue가 없으면 수신과 처리를 분리할 수 없다

기존 방식 vs 202+Queue 방식 — 한눈 비교

기존 방식 — 즉시 처리

VASP 이벤트 수신

비즈니스 로직 실행

DB 저장

VASP에 응답

202+Queue 방식 — 비동기 분리

VASP 이벤트 수신

Queue 적재

즉시 202 반환

비동기

Consumer 이벤트 읽기

비즈니스 로직 실행

DB 저장 + XACK

장애 시 동작 비교 — 어디서 차이가 나는가

시나리오	기존 방식 (즉시처리)	202+Queue 방식
DB 장애	이벤트 유실 ✗	Queue 잔류 → 복구 ✓
VASP 타임아웃	재전송 → 중복 처리 ✗	202 즉시 응답 → 재전송 없음 ✓
Consumer 장애	없음 (단일 처리)	Queue 잔류 → 재시작 후 재수신 ✓
처리 속도 저하	VASP 타임아웃 연쇄 ✗	수신 속도 일정 유지 ✓

202 Accepted — 이 숫자의 정확한 의미

200 OK
처리 완료. 결과를 지금 돌려줌.

VS

202 Accepted
요청 접수 완료. 처리는 나중에.

202 = 처리를 약속한 것이지 완료한 것이 아님 — Webhook에 가장 적합한 응답 코드

VASP 입장

나는 202를 받았다
→ 성공으로 간주
→ 재전송 하지 않음

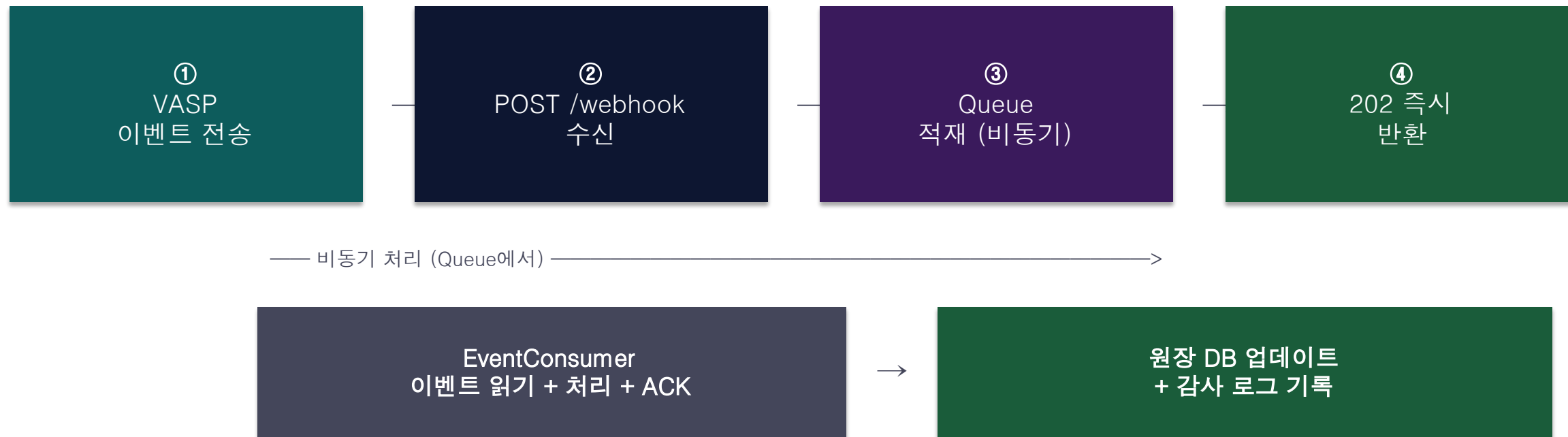
우리 시스템 입장

이벤트 접수만 했음
→ 처리는 비동기
→ DB에 아직 안 썼음

결과

VASP: 재전송 없음 ✓
시스템: 부담 없음 ✓
내구성: Queue가 보장 ✓

202 패턴 — 이벤트 처리 흐름



202 반환 시점: Queue 적재 직후 — DB 처리와 무관하게 즉시

Queue 적재 실패 시 — 어떻게 할 것인가

옵션 A 500 반환

VASP가 재전송
→ VASP 재전송 보장 필요
→ 멍등성 구현 필수

옵션 B 로컬 파일 폴백

Queue 장애 시 파일 큐
→ 복잡도 증가
→ 소규모 시스템에서 가끔 사용

옵션 C Queue HA 구성

Redis Sentinel/Cluster
→ Queue 자체 고가용성 확보
→ 교보 Phase 1 권장

Queue 적재도 실패할 수 있다 — 이 경우의 정책을 명확히 해야 한다

교보 Phase 1: Redis 단일 노드 → Queue 장애 시 500 반환 후 VASP 재전송 의존 (MVP 범위)

응답 순서 결정 — 적재 먼저? 응답 먼저?

잘못된 순서 A — 응답 먼저

202 응답 반환

Queue 적재 시도

Queue 장애 발생!

이벤트 유실 — VASP는 성공으로 인식

올바른 순서 B — 적재 먼저

Queue 적재 성공 확인

202 응답 반환

(Queue 장애 시 500 반환)

결론: Queue 적재 성공 확인 → 202 응답. 적재 실패 → 500. VASP가 재전송하도록 유도

202 패턴의 전제 조건 3가지

Queue 내구성 필수

이벤트가 Queue에 들어가면
안전하게 보관되어야 함
→ Redis Streams 채택 이유

Consumer 재처리 가능

처리 실패 시 Queue에서
다시 읽을 수 있어야 함
→ At-least-once 보장

멱등성 구현 필수

동일 이벤트 2회 처리해도
결과가 같아야 함
→ S9에서 상세 학습

이 3가지를 Redis Streams + EventConsumer + 멱등성 체크로 구현

202 패턴은 단순한 숫자가 아니라 — 수신/처리 분리를 위한 아키텍처 결정이다

VASP 관점 — 그들은 무엇을 기대하는가

VASP = Virtual Asset Service Provider = 외부 가상자산 시스템 (거래소, 지갑 등)

VASP의 기대

빠른 응답 (타임아웃: 3~10초)
성공 확인 (2xx 수신)
재전송 조건 명확

VASP의 재전송 조건

응답 없음 → 타임아웃 후 재전송
5xx → 서버 오류로 재전송
2xx → 성공, 재전송 없음

202의 효과

VASP: 2xx 수신 → 성공
재전송 없음 → 중복 없음
우리 시스템: 여유 있게 처리

202 = VASP와 우리 시스템 사이의 계약 — "받았어. 처리는 내가 알아서 할게"

202 패턴 적용 범위 — 언제 쓰고 언제 쓰지 않는가

202 사용 ✓

- Webhook 이벤트 수신
- 배치 처리 요청
- 결과가 즉시 필요 없는 작업
- 처리 시간이 긴 작업

200 사용 ✓

- 결제 승인 요청
- 데이터 조회
- 실시간 검증 필요 작업
- 응답 결과에 의존하는 작업

202 잘못된 사용 ✗

- 처리 결과를 바로 반환해야 할 때
- 후속 처리 없이 202만 반환
- Queue 없이 202 반환
- 멍등성 없이 재처리 허용

Section 요약 — 202 패턴 핵심

202 패턴 = 수신과 처리의 경계를 명확히 하는 아키텍처



즉시 처리 위험

DB 장애 → 유실 / VASP 타임아웃 → 재전송 폭탄



202 즉시 반환

Queue 적재 성공 확인 → 202 응답 → VASP 재전송 없음



Queue 중간 단계

내구성 + 독립 확장 + 재처리 가능성 확보



전제 조건

Queue 내구성 + 재처리 + 멍등성 — 세 가지 모두 필요

WebhookReceiver — 역할과 책임 범위

WebhookReceiver 핵심 원칙: 단순하고 빠르게

담당 ✓

- POST /webhook 라우트
- 이벤트 형식 기본 검증
- Queue 적재 요청
- 202 즉시 반환

비담당 ✗

- 비즈니스 로직 실행
- DB 직접 접근
- 원장 업데이트
- 복잡한 데이터 변환

WebhookReceiver가 DB에 접근하는 순간 — 즉시 처리의 위험으로 돌아간다

POST /webhook — 라우트 구조

엔드포인트 설계 결정사항

경로	POST /webhook	VASP와 계약 시 이 경로를 등록
인증	HMAC-SHA256 서명 헤더	S8에서 구현 — 지금은 구조만
응답 성공	202 + { received: true }	처리 완료가 아님 명시적 표현
응답 실패	400 (검증 실패) / 500 (Queue 오류)	VASP 재전송 트리거
DB 접근	없음	응답 속도 보장

VASP 이벤트 구조 — 무엇이 들어오는가

NFT 발행 이벤트 — VASP가 전송하는 데이터 구조

eventType	string	NFT_ISSUED / NFT_BURNED / TRANSFER	이벤트 종류 식별
contractAddress	string	0x... (KyoboNFT 컨트랙트)	어느 컨트랙트의 이벤트
tokenId	bigint	1~N	NFT 식별자
recipient	string	0x... (수령인 지갑)	원장 업데이트 대상
blockNumber	number	최신 블록 번호	Finalized 판단에 사용
txHash	string	0x... (트랜잭션 해시)	중복 처리 방지 키
timestamp	number	Unix timestamp	감사 로그용

이벤트 검증 수준 — 무엇을 WebhookReceiver에서 하는가

WebhookReceiver에서 할 것

- ✓ 필수 필드 존재 여부 (eventType, txHash)
- ✓ eventType 허용 값 (enum 체크)
- ✓ 기본 형식 검사 (address 길이 등)
- ✓ HMAC 서명 검증 (S8)

Consumer에서 할 것 (비동기)

- ↔ 컨트랙트 주소 유효성 검증
- ↔ blockNumber Finalized 여부
- ↔ 멍등성 체크 (txHash 중복)
- ↔ 비즈니스 로직 실행

원칙: Webhook Receiver는 이벤트를 "받을 자격"만 검사 — "처리 가능성"은 Consumer

QueueService — 적재 인터페이스 설계

WebhookReceiver → QueueService 호출 구조

QueueService

enqueue(event: WebhookEvent)
→ Redis Streams XADD
→ 성공 시 messageId 반환
→ 실패 시 Error throw



Redis Streams

XADD [eventStream] [*] [field] [value]
→ auto-generated ID 반환
→ 메모리에 영구 저장
→ Consumer Group 대기

QueueService는 Redis Streams 세부 구현을 숨기는 추상화 레이어 — S6에서 상세 구현

WebhookReceiver: enqueue 성공 → 202 / 실패 → 500

응답 구조 — 202와 에러 응답 설계

성공 응답 — 202 Accepted

HTTP/1.1 202 Accepted

Content-Type: application/json

```
{ "received": true,  
  "messageId": "1234567890-0" }
```

messageId: Redis가 발급한 Streams ID
(디버깅·추적용)

에러 응답

400

Bad Request

필수 필드 누락 / 잘못된
형식

401

Unauthorized

HMAC 서명 검증 실패
(S8)

500

Server Error

Queue 적재 실패

스켈레톤 — 오늘 채울 파일 위치

교보 프로젝트 스켈레톤 구조 — S5 관련 파일

src/api/webhook/	WebhookController.ts	POST /webhook 라우트 정의
src/api/webhook/	WebhookReceiver.ts	이벤트 수신 + Queue 적재 + 202 반환
src/queue/	QueueService.ts	enqueue() 인터페이스 정의 (stub)
src/types/	WebhookEvent.ts	이벤트 타입 정의
test/webhook/	webhook.test.ts	202 응답 + Queue 적재 테스트

stub = 껍데기만 있는 파일. S5에서 WebhookReceiver를 완성하고 QueueService는 stub 상태 유지

S5 완료 기준

이것을 할 수 있어야 S5 완료

✓	설계 이해	202 패턴 설계 이해 + 즉시 처리 위험 설명 가능
✓	코드 구조	WebhookReceiver 라우트 구조 완성
✓	응답 확인	POST /webhook 호출 시 202 응답 반환 확인
✓	Queue 연결	enqueue() 호출 후 Queue에 이벤트 적재 확인
✓	에러 처리	Queue 실패 시 500 반환 확인

다음 세션 (S6): Redis Streams 내부 구조와 At-least-once 처리 보장 원리

COINCRAFT RESEARCH

실습

202 패턴 구현 — POST /webhook + Queue 적재 + 202 응답

S5 실습 | 30분

COINCRAFT
ACADEMY

실습 개요 — 오늘 만들 것

목표: VASP 이벤트 수신 → 202 즉시 응답 구현

1단계

WebhookEvent 타입 정의

eventType / txHash / tokenId / recipient / blockNumber

2단계

QueueService stub 확인

enqueue() 인터페이스만 — 실제 구현은 S6

3단계

WebhookReceiver 구현

수신 → 검증 → enqueue() → 202 반환

4단계

테스트 실행

정상 이벤트 → 202 / 필드 누락 → 400 / Queue 실패 → 500

실습 1 단계 — WebhookEvent 타입 정의

src/types/WebhookEvent.ts

eventType

: 'NFT_ISSUED' | 'NFT_BURNED' | 'TRANSFER'

contractAddress

: string // 0x...

tokenId

: bigint

recipient

: string // 0x...

blockNumber

: number

txHash

: string // 중복 처리 방지 키

timestamp

: number // Unix timestamp

txHash가 핵심 — 멍등성 체크의 기준값 (S9에서 활용)

실습 2단계 — QueueService Stub 확인

src/queue/QueueService.ts — 지금은 stub만 확인

enqueue(event)

이벤트를 Queue에 적재

→ messagedId 반환 (string)

interface 정의

IQueueService

→ WebhookReceiver는 인터페이스만 의존

실제 구현

RedisSteamsQueueService

→ S6에서 구현

의존성 역전 원칙: WebhookReceiver는 IQueueService에 의존 → 구현체 교체 가능

실습 3단계 — WebhookReceiver 구현

src/api/webhook/WebhookReceiver.ts

①	이벤트 수신	POST body 파싱
②	기본 검증	eventType, txHash 필수 확인
③	Queue 적재	queueService.enqueue(event) 호출
④	202 반환	{ received: true, messageId }
⑤	에러 처리	검증 실패 → 400 / Queue 실패 → 500

⚠ 중요: Queue 적재 성공을 확인한 후 202 반환 — 순서 절대 변경 금지

실습 4단계 — 테스트 시나리오

test/webhook/webhook.test.ts — 3가지 시나리오

테스트 1

정상 이벤트 수신
모든 필드 포함 이벤트 전송

→ 202 + messageId 확인

테스트 2

필수 필드 누락
txHash 없는 이벤트 전송

→ 400 Bad Request 확인

테스트 3

Queue 장애 시뮬레이션
QueueService mock → 에러 throw

→ 500 Server Error 확인

테스트 실행 + 결과 확인

실습 실행 순서		
터미널 1	서버 기동	<code>npm run dev</code>
터미널 2	테스트 실행	<code>npm test -- webhook.test</code>
결과 확인	테스트 통과	● webhook: POST /webhook 202 ✓

테스트 3개 모두 통과 = S5 완료. 다음: S6 Redis Streams 내부 구조

실습 함정 포인트 — 자주 만나는 실수

함정 1	Queue 적재 전에 202 반환	enqueue() 실패해도 VASP에 성공 전달 → 이벤트 유실	✓ 큐 적재 성공 확인 후 202
함정 2	WebhookReceiver에서 DB 접근	DB 장애 시 이벤트 전체 차단 → 즉시 처리와 동일	✓ DB는 Consumer에서만
함정 3	에러 시 202 반환	Queue 실패를 성공으로 숨김 → 이벤트 유실	✓ 실패는 명확히 500

이 실수들은 실제 현업에서도 자주 발생한다

COINCRAFT RESEARCH

정리

오늘 배운 것 · 핵심 원칙 · 다음 세션 예고

S5 마무리

COINCRAFT
ACADEMY

S5 핵심 요약 — 3줄로 기억하기

1. 즉시 처리는 위험하다

DB 장애 → 이벤트 유실 / VASP 타임아웃 → 재전송 폭탄

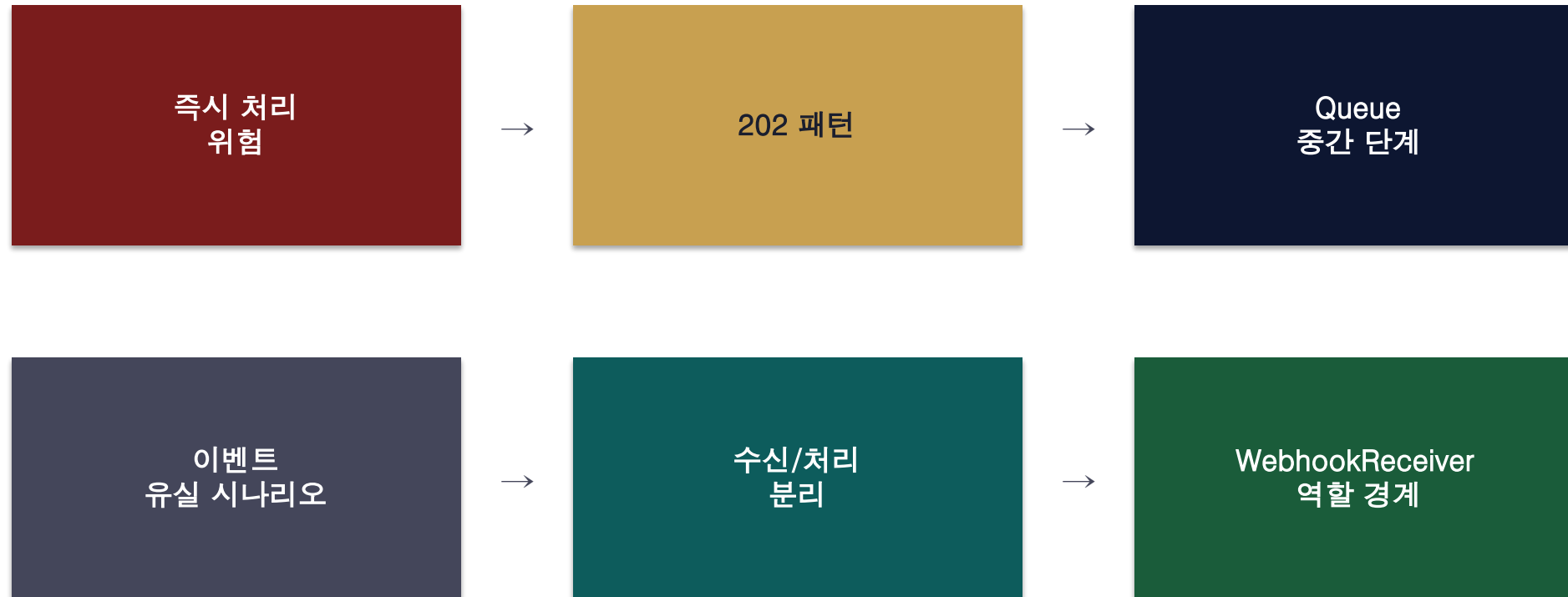
2. 202 패턴 = 수신과 처리를 분리하는 계약

Queue 적재 성공 → 202 즉시 반환 → VASP 재전송 없음

3. WebhookReceiver = 단순하고 빠르게

DB 접근 없음 / 비즈니스 로직 없음 / 오직 수신 + 적재 + 202

오늘 배운 개념 지도



다음 세션 — S6 Redis Streams 내부 구조

S5에서 쓴 QueueService → S6에서 Redis Streams로 실제 구현

Pub/Sub vs Queue vs Streams

왜 Streams를 선택했는가

Stream Entry 구조

XADD / XREADGROUP / XACK

Consumer Group

분산 처리 + 메시지 분배

At-least-once

PEL + 장애 시 재처리 메커니즘

수평 확장

Consumer 인스턴스 추가 = 처리량 증가

S6 완료 후: QueueService가 실제 Redis Streams와 연결됨

S5

M2 전체 맥락 — S5 완료 후 위치

S5
이벤트 수신
202 패턴
✓ 완료

S6
Redis Streams
구조

S7
Streams
CLI 실습

S8
Webhook 보안
병렬 처리

S9
처리 순서
불변 규칙

S10
EventConsumer
먹등 처리

S11
DLQ
장애 격리

S12
Finalized
복원력

S5 완료 ✓ — 파이프라인의 입구를 완성했다

금융 시스템에서 이벤트 유실의 실제 비용

이벤트 하나가 유실되면 무슨 일이 생기는가?

사용자 피해

NFT 미발행 → 자산 누락
고객 민원 → 콜센터 과부하
보상 처리 → 추가 비용

운영 피해

원장 불일치 → 감사 실패
재처리 불가 → 수동 복구
시스템 신뢰 하락

규제 리스크

가상자산법 위반 가능성
금융당국 제재 리스크
고객 데이터 무결성 위반

비동기 분리 + Queue 내구성 = 이 모든 리스크를 사전 차단

이벤트 주도 아키텍처 — DMZ 파이프라인의 철학

DMZ 파이프라인 = 이벤트 주도 아키텍처(EDA)의 금융 적용

느슨한 결합

발신자와 수신자가 서로 모름
VASP: URL만 알면 됨
우리: 처리 방식 자유

비동기 처리

이벤트 발생 ≠ 즉시 처리
큐에 쌓이고 순서대로
처리 속도 독립 제어

확장 가능성

Consumer 수 = 처리량
추가/제거 자유
다운타임 없는 확장

감사 가능성

모든 이벤트 기록
재처리 가능
감사 추적 완전

202 패턴은 EDA의 첫 번째 구현 — 이벤트를 받아서 Queue에 넣는 것

동기 시스템 vs 비동기 시스템 — 장단점 비교

동기 시스템 (즉시 처리)

- ✓ 구현 단순
- ✓ 디버깅 쉬움
- ✓ 즉각적 피드백
- ✗ 장애 전파
- ✗ 확장 어려움
- ✗ 타임아웃 취약

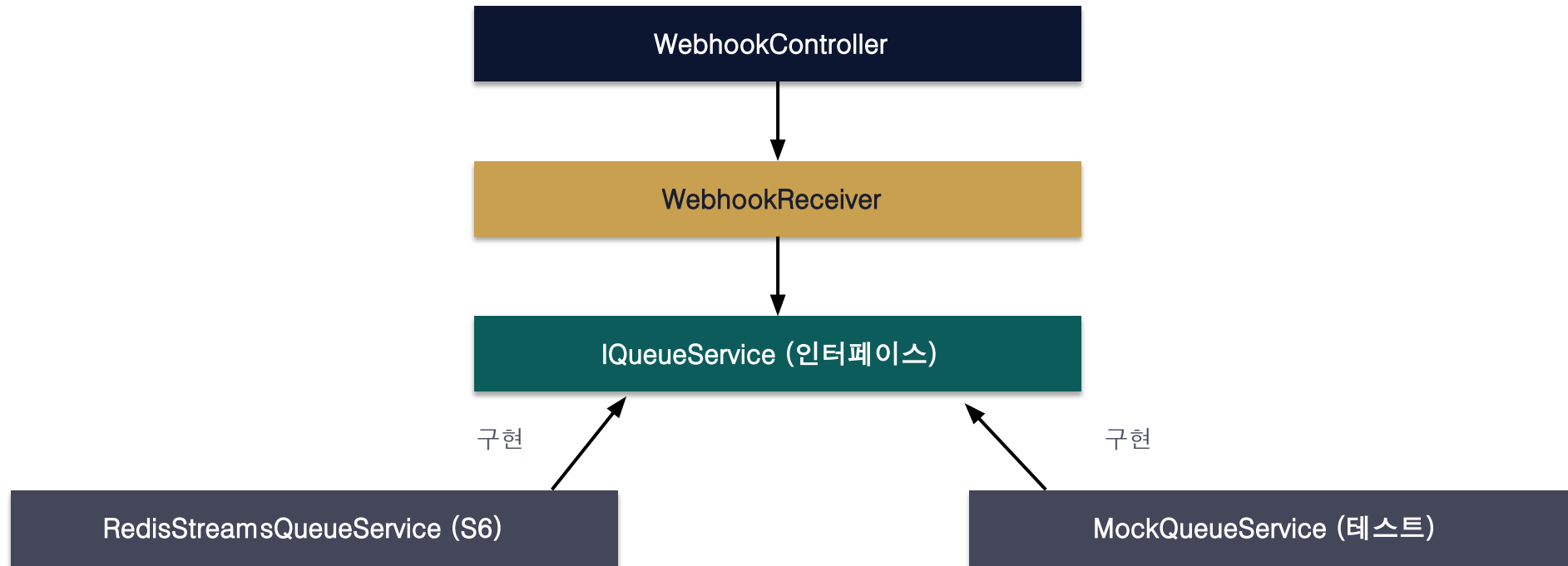
비동기 시스템 (202+Queue)

- ✓ 장애 격리
- ✓ 독립 확장
- ✓ 재처리 가능
- ✓ VASP 부담 없음
- ⚠ 구현 복잡
- ⚠ 디버깅 추적 필요
- ⚠ 최종 일관성

금융 시스템 = 내구성 우선 → 비동기 시스템 선택이 기본값

스켈레톤 의존 방향 — S5 완성 후 상태

S5 완료 시점 의존 관계 — 화살표 방향이 의존 방향



WebhookReceiver는 IQueueService에 의존 — 구현체(Redis vs Mock)와 무관

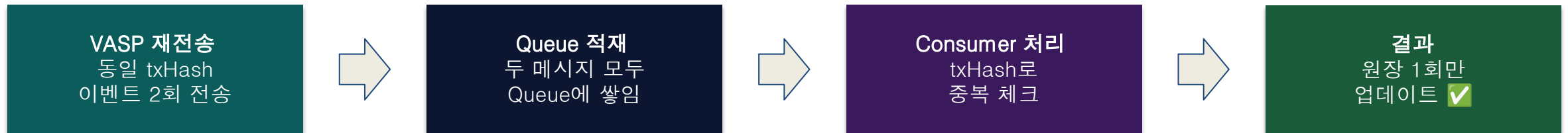
실습 시작 전 환경 점검

실습 전 체크리스트 — 이 항목이 준비돼야 실습 가능

✓	Node.js + npm	node --version → v18+	터미널에서 확인
✓	프로젝트 클론	kyobo-digital-asset-platform 폴더 존재	git clone 완료 상태
✓	의존성 설치	npm install 완료	node_modules 폴더 존재
✓	TypeScript 컴파일	npx tsc --noEmit 에러 없음	타입 오류 없어야 시작 가능
✓	테스트 환경	npm test 기본 실행 확인	Jest 또는 Vitest 동작

미리 보기 — Idempotency Key (S9 사전 이해)

Idempotency Key = 동일한 요청을 여러 번 처리해도 결과가 같게 만드는 키



txHash를 WebhookEvent에 반드시 포함시킨 이유 — 지금 심어두는 멍등성 씨앗

S5에서 만든 txHash가 S9에서 핵심 역할을 한다

S9에서 ON CONFLICT DO NOTHING으로 완성 — 지금은 txHash 필드 존재만 이해

운영 관점 — 무엇을 모니터링해야 하는가

WebhookReceiver 운영 시 핵심 메트릭

수신 QPS	초당 Webhook 수신 요청 수	급증 시 VASP 재전송 폭탄 의심
202 비율	전체 요청 대비 202 응답 비율	비율 하락 = Queue 장애 신호
400 비율	잘못된 형식 이벤트 비율	VASP 연동 오류 또는 버전 불일치
500 비율	Queue 적재 실패 비율	Redis 장애 즉시 알림 트리거
응답 지연	p95/p99 응답 시간	100ms 초과 시 Queue 병목 조사

예상 질문 & 답변

Q

Queue 적재도 실패할 수 있는데, 그러면 이벤트 유실 아닌가요?

A

맞다. Queue 자체 HA(Redis Sentinel/Cluster)로 해결. MVP에서는 500 반환 후 VASP 재전송 의존. S6에서 Redis 구성 다룬다.

Q

202 응답 후 Consumer가 영영 처리 안 하면?

A

DLQ(Dead Letter Queue)가 답. S11에서 3회 실패 → DLQ → 알림 → 수동 재큐잉 구현.

Q

왜 Redis를 쓰나요? Kafka 같은 것도 있잖아요?

A

교보 Phase 1 규모에서 Redis Streams가 적합. Kafka는 초대형 처리량에 적합하지만 운영 복잡도가 높음.

핵심 용어 정리

S5 핵심 용어 — 다음 세션에서도 계속 쓰인다

202 Accepted

요청 접수 완료. 처리는 비동기.

WebhookReceiver

이벤트 수신 + 202 응답 담당 컴포넌트

QueueService

이벤트를 Queue에 적재하는 추상화 레이어

At-least-once

최소 1번 처리 보장. 중복 가능. 멱등성 필요.

Idempotency

동일 요청 N번 처리 = 결과 동일

EDA

Event-Driven Architecture. 이벤트 주도 아키텍처

DLQ

Dead Letter Queue. 처리 실패 이벤트 격리 큐

수고하셨습니다

S5 — 온체인 이벤트 수신 설계 즉시 처리의 위험과 비동기 분리

오늘 완성한 것

WebhookReceiver ✓
202 패턴 ✓
수신/처리 분리 ✓

다음 세션 (S6)

Redis Streams 내부 구조
At-least-once 원리
Consumer Group

질문 · 피드백 환영합니다